

Deep Learning

Architectures CNN's

Research Depth and Industry Skills

By:

Dr. Zohair Ahmed



www.youtube.com/@ZohairAI

Subscribe



www.begindiscovery.com

ConvNets vs. Traditional Methods

- Image classification means:
 - *Given an image, the computer must decide which category it belongs to.*
- Traditionally, this is done in **two separate steps**:
 1. **Feature Extraction**
 2. **Classification**
- ConvNets (CNNs) changed this completely.



Traditional Image Classification Pipeline

- **Step 1: Feature Extraction**
- Before deep learning, computers couldn't directly understand pixels. So experts hand-crafted features, ways of describing the image, such as:
- **SIFT** (detects interesting points)
- **HOG** (captures edges and directions)
- **LBP** (detects texture)
- These features are computed **without knowing the image labels**, this is *unsupervised*.
- The downside?
 - The features **do not learn** what makes *cats look like cats* or *cars look like cars*.
 - They just extract generic patterns.

Traditional Image Classification Pipeline

- **Step 2: Classification**
- Once features are extracted, a classifier is trained using labels.
- Common traditional classifiers:
 - SVM
 - Logistic Regression
 - Decision Trees
 - Random Forests
- So the pipeline looks like:
- Image → Feature Extraction → Classifier → Category



Problems With Traditional Pipeline

1. Features are not optimized for the task

- Handcrafted features cannot adapt.
- They might fail to capture what distinguishes one class from another.

2. Accuracy is limited

- Even if you use the best classifier (SVM, etc.), performance is limited by **how good the features are**.

3. Requires a lot of manual engineering

- Researchers often combined multiple features (SIFT + HOG + LBP, etc.).
- **This involved:**
- many heuristics
- domain-specific tweaks
- lots of manual trial + error

4. Not how humans learn

- We don't manually define “edge detectors” or “texture filters” when learning to recognize objects.
- We learn directly from examples.

ConvNets (CNNs): The Modern Approach

- ConvNets solve the biggest weakness of traditional methods:
- **They learn features automatically from data**
- **Feature extraction and classification happen together**
- So the pipeline becomes:
- Image → ConvNet → Category
- No separate hand-crafted feature extraction module.



Why ConvNets Work Better

1. Automatic Feature Learning

- ConvNets *learn* the most useful features for the task using labeled data.
- They learn:
 - edges in early layers
 - shapes in middle layers
 - objects in deeper layers
 - All automatically.

2. Parameter Sharing

- Instead of every neuron being fully connected:
- a small filter (like 3×3) slides over the image
- this filter's weights are shared across the entire

image

- This makes ConvNets:
 - **efficient**
 - **less prone to overfitting**
 - **trainable on large datasets like ImageNet**

3. Sparse Connectivity

- Each neuron connects only to a small part of the image (local receptive field), not the entire image.

This reduces the number of parameters dramatically.

Modern CNN Architectures

- Modern CNNs use smart, repeatable blocks (like ResNet blocks, Inception modules).
- Architecture evolution shifted from:
- simple stacked layers → **innovative, modular building blocks**
- This allows:
 - better accuracy
 - deeper networks
 - faster training

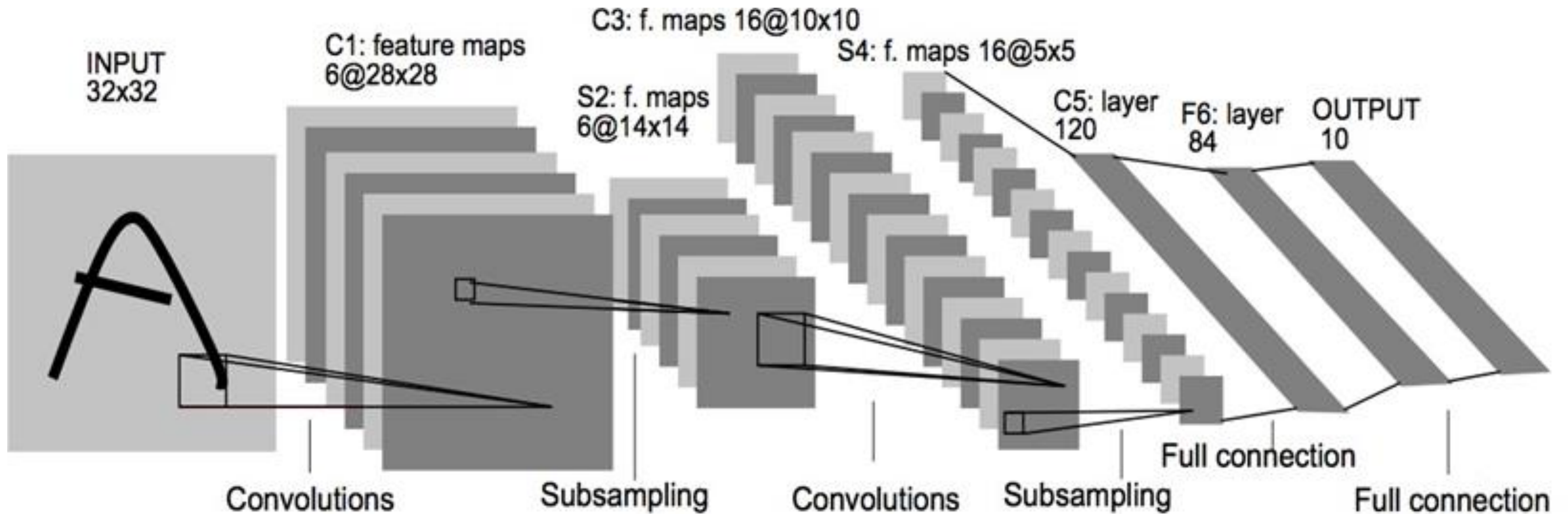


LeNet-5 (1998)

- **Download Paper**
 - http://vision.stanford.edu/cs598_spring07/papers/Lecun98.pdf
- **Purpose:** Recognizing handwritten digits (e.g., postal zip codes)
- **Key Ideas**
- LeNet-5 was one of the **first successful CNN architectures**.
- Introduced concepts that all modern CNNs still use:
 - Convolution
 - Subsampling (pooling)
 - Feature maps
 - End-to-end learning
- Subsampling means reducing the size of the feature maps while keeping the important information.



LeNet-5 (1998)



LeNet-5 (1998): Architecture Highlights

- Convolution layers use **only part of the previous layer's channels** → reduces computation.
- Subsampling uses **average pooling** (not max pooling).
- Total parameters: **~60,000** (very small compared to today's networks).
- **Why It Matters**
 - LeNet-5 is historically important because it **introduced the foundation** of modern CNNs.
 - Even modern architectures are built on these same ideas.

LeNet-5 Architecture

1. Input Layer (32×32 Image)

- The network takes a **32×32 grayscale image** as input (MNIST digits).
- MNIST images are originally **28×28**, so **padding is added** to make them 32×32.

2. First Convolutional Layer (C1)

- **6 filters**, each of size **5×5**, stride = 1
- Produces output feature maps of size: **28×28×6**
- **Activation:** Sigmoid (original) or ReLU (modern)

3. First Subsampling Layer (S2 – Pooling)

- **Average pooling**, filter **2×2**, stride = 2
- Output size becomes: **14×14×6**
- **Purpose:** Reduce spatial dimensions while keeping useful information

4. Second Convolutional Layer (C3)

- **16 filters**, each **5×5**
- **Note:** Not all filters connect to all previous maps (reduces computation and adds sparsity)
- Output size: **10×10×16**
- **Activation:** Sigmoid/ReLU

LeNet-5 Architecture

5. Second Subsampling Layer (S4 – Pooling)

- Average pooling, filter 2×2
- Output size: $5 \times 5 \times 16$

6. Fully Connected Layer (C5)

- Flatten input: $5 \times 5 \times 16 = 400$ neurons
- Fully connected to **120 neurons**
- **Activation:** Sigmoid/ReLU

7. Fully Connected Layer (F6)

- Fully connected to **84 neurons**
- **Activation:** Sigmoid/ReLU

8. Output Layer

- Fully connected to **10 neurons** (digits 0–9)
- **Activation:** Softmax for classification

LeNet-5 Architecture

Layer	Type	Output Size
Input	Image	$32 \times 32 \times 1$
C1	Conv ($6 \times 5 \times 5$)	$28 \times 28 \times 6$
S2	Avg Pooling	$14 \times 14 \times 6$
C3	Conv ($16 \times 5 \times 5$)	$10 \times 10 \times 16$
S4	Avg Pooling	$5 \times 5 \times 16$
C5	Fully Connected	120
F6	Fully Connected	84
Output	Softmax	10 classes



AlexNet (2012)

- **Download:**

- [https://proceedings.neurips.cc/paper/4824-
imagenet-classification-with-deep-
convolutional-neural-networks.pdf](https://proceedings.neurips.cc/paper/4824-
imagenet-classification-with-deep-
convolutional-neural-networks.pdf)

- **Purpose:** ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

Key Innovations

- **ReLU activation**

- Faster training than sigmoid/tanh
- Avoids vanishing gradients

- Trains **5× faster** with similar accuracy.

- **Dropout**

- Introduced dropout for FC layers
- Helps reduce overfitting
- Randomly drops neurons with probability p

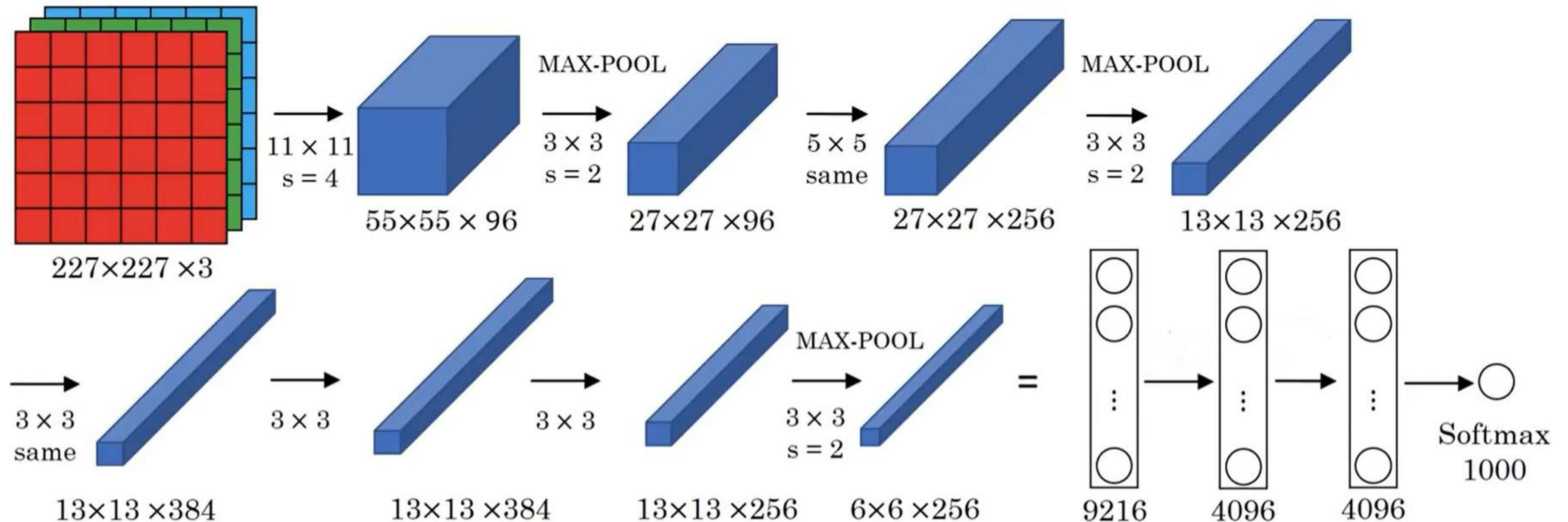
- **GPU training**

- Trained on two GPUs due to large size
- Pushed deep learning into mainstream.

AlexNet (2012)

- **Architecture Overview**
- Input: **$227 \times 227 \times 3$** RGB image
- First Conv layer:
 - 96 filters
 - Size: **11×11**
 - Stride: 4
- Output shrinks to **55×55**
- Max Pooling:
 - 3×3 filter
 - Stride 2
- Fully Connected layers:
 - FC1: 9216 neurons
 - FC2: 4096 neurons
 - FC3: 4096 neurons
- Softmax at the end with **1000 classes**
- Total parameters: **~60 million**
- **Why It Matters**
- AlexNet showed the world that **deep CNNs + GPUs = massive accuracy boost.**
- It essentially started the modern deep learning revolution in computer vision.

AlexNet (2012)



VGG (2014)

- **Download:**

- <https://www.robots.ox.ac.uk/~vgg/publications/2015/Simonyan15/simonyan15.pdf>

- **By:** Visual Geometry Group (VGG), University of Oxford

- **Purpose:** Improve CNN depth while keeping architecture simple

- **Key Innovations**

- **Use only 3×3 convolution filters**

- Replaced large filters in AlexNet (11×11, 5×5)
- Stacking multiple small filters gives **same receptive field** as larger ones
- But with **fewer parameters** and **more expressive non-linearities**

- **Deep but simple**

- VGG-16 has 16 layers

- VGG-19 has 19 layers

- Very uniform and easy to understand architecture

- **SAME padding + Max Pooling**

- Every pooled output reduces size by half
- Filters always 3×3 , stride = 1
- Pooling layers always 2×2 , stride = 2

- **Architecture Pattern**

- [Conv 3×3] → [Conv 3×3] → Pool

- [Conv 3×3] → [Conv 3×3] → Pool

- ...

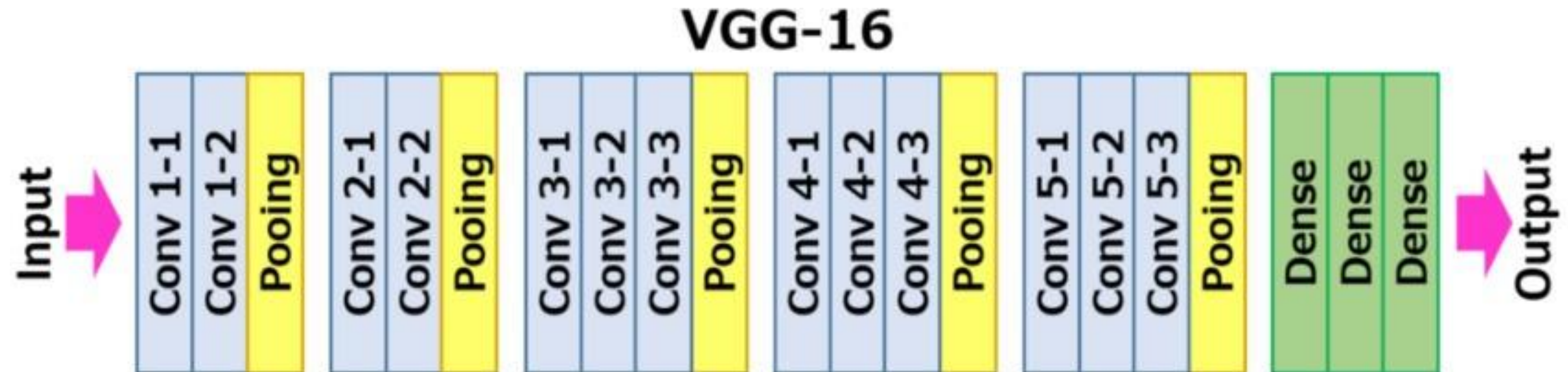
- → FC → FC → Softmax

Why It Matters

- VGG proved that **depth improves performance**. It became extremely popular due to its simplicity.



VGG (2014)



What Are Pre-Trained Models?

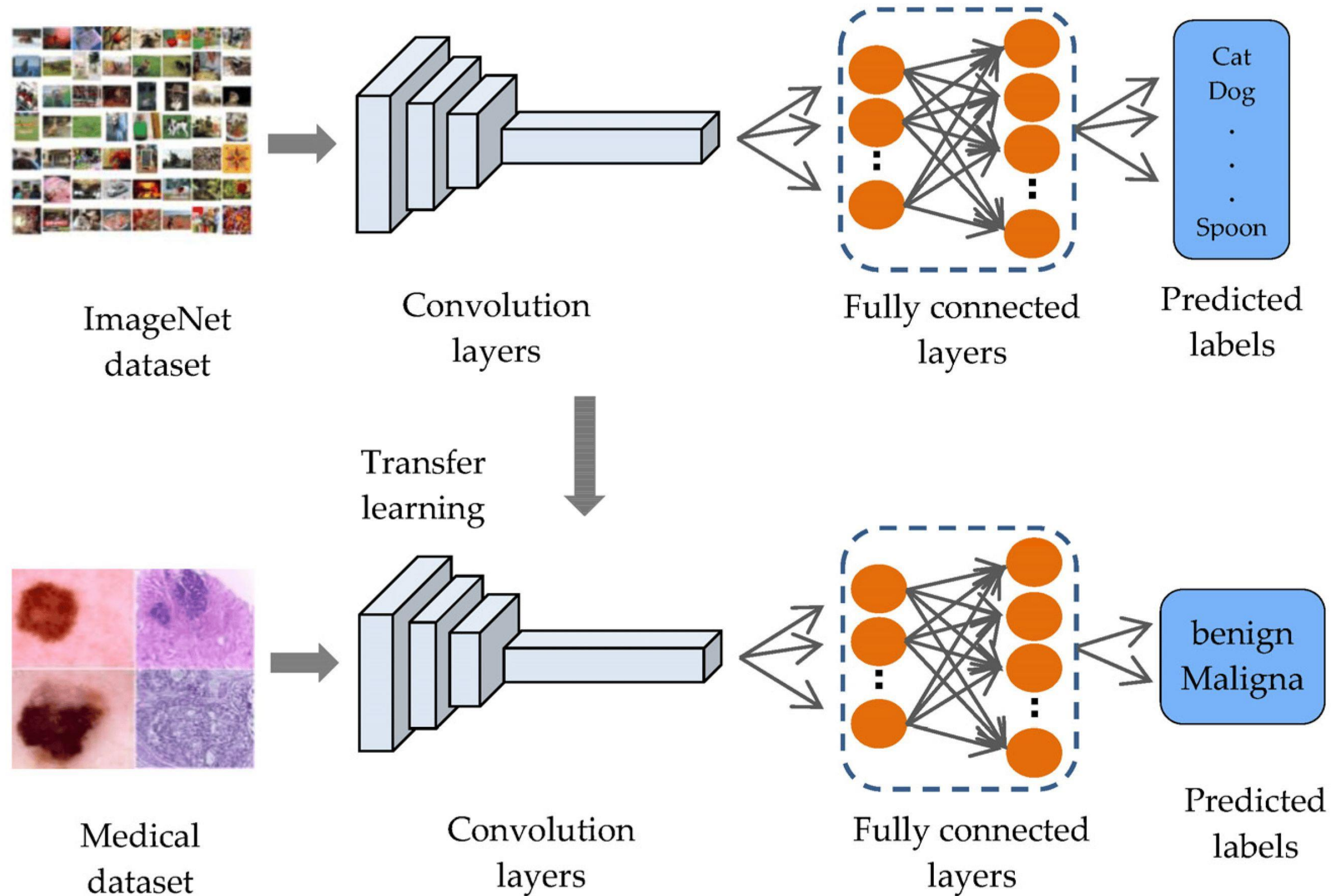
- Pre-trained models are **deep learning models that have already been trained on a very large dataset**, usually ImageNet (1.2 million images, 1000 classes).
 - MobileNet
- These models already “know” how to detect:
 - edges
 - textures
 - shapes
 - objects
- Examples include:
 - VGG16
 - ResNet
 - AlexNet
 - Inception
- So we don't need to train them from scratch.

What is Transfer Learning?

- **Transfer Learning** means:
 - *Take a pre-trained model*
 - *Reuse it on a new (usually smaller) dataset*
 - *Fine-tune it for your specific task*
- This works because early layers in CNNs learn **general features** (edges, corners, textures).
- These features apply to almost all image tasks.



What is Transfer Learning?



Why Use Transfer Learning?

- **1. Avoids Training from Scratch**
 - Training big networks (like VGG or AlexNet) from scratch requires millions of images.
Small datasets will fail → results in **overfitting**.
- **2. Reduces Training Time**
 - Model has already learned basic features.
 - Only last few layers need training → **much faster**.
- **3. Prevents Overfitting**
 - With few images, training a deep model is risky.
 - Transfer learning provides a strong starting point.
- **4. Improves Accuracy**
 - Models trained on ImageNet generalize well and boost performance on smaller datasets.



When Should You Use Transfer Learning?

- **Recommended When:**

- Dataset is **small** (a few thousand images)
- Dataset is **similar to ImageNet** (cars, cats, fruits, etc.)
- No computing resources to train from scratch

- **Not Recommended When:**

- Dataset is **completely different** from ImageNet
- (medical images, X-rays, satellite images)
- Dataset is **very large** (>1 million images)



How to Use Transfer Learning?

- There are **two main approaches**:

A. Feature Extraction (Freezing Layers)

- Freeze all pre-trained convolutional layers
 - No weight updates
 - Remove old fully connected layers
 - Add **new classifier layers**
 - Train only the new layers
- **Use this when:**
 - small dataset
 - similar to ImageNet

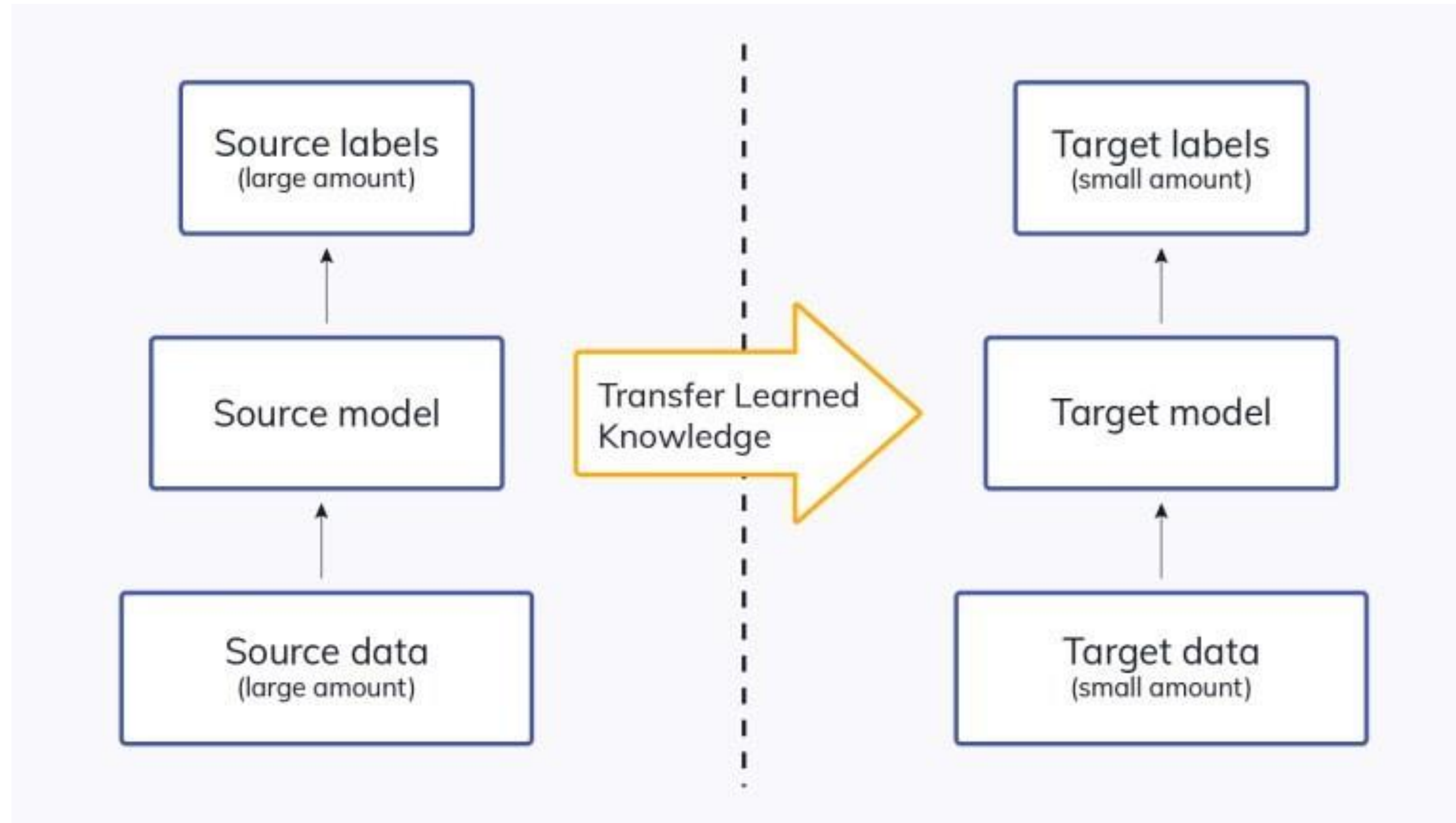
B. Fine-Tuning (Unfreezing Some Layers)

- Freeze earlier layers
 - Unfreeze **some deeper convolutional layers**
- Train them **together** with the new classifier
 - **Use this when:**
 - dataset is large
 - dataset is different from ImageNet

Using VGG16 for a Custom Dataset

- **Step 1: Load Pretrained VGG16 Model**
 - With weights = "ImageNet" **extraction)**
 - **or**
 - Unfreeze some layers and fine-tune (fine-tuning)
- **Step 2: Remove Last FC Layers**
 - Remove 1000-class classifier
- **Step 3: Add Your Own Layers**
 - Example:
 - Dense(128), Dense(10) for your custom classes
- **Step 4: Compile and Train**
 - Train only the new top layers (feature

Transfer Learning



Transfer Learning

Freeze or fine-tune?

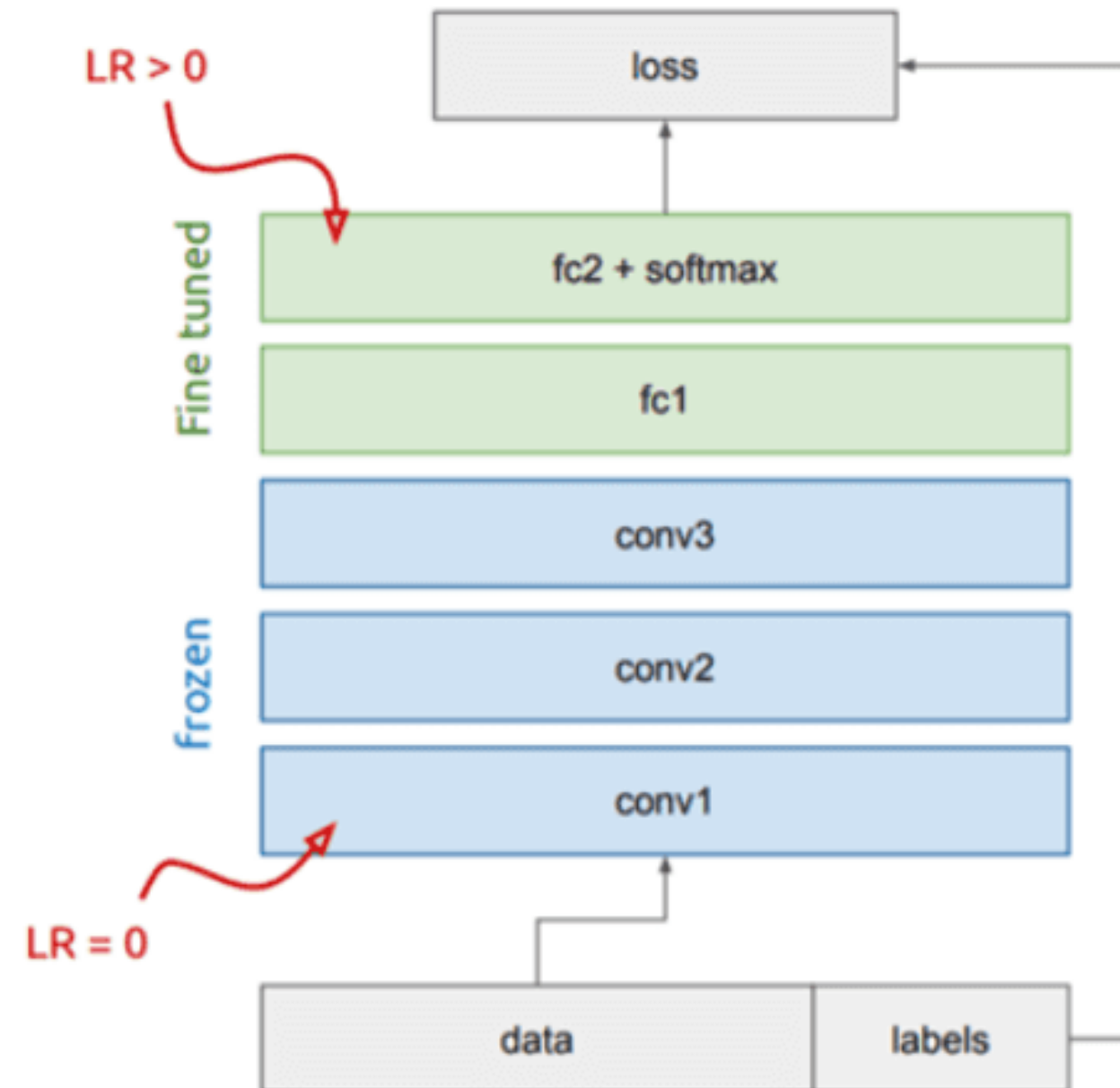
Bottom n layers can be frozen or fine tuned.

- **Frozen:** not updated during backprop
- **Fine-tuned:** updated during backprop

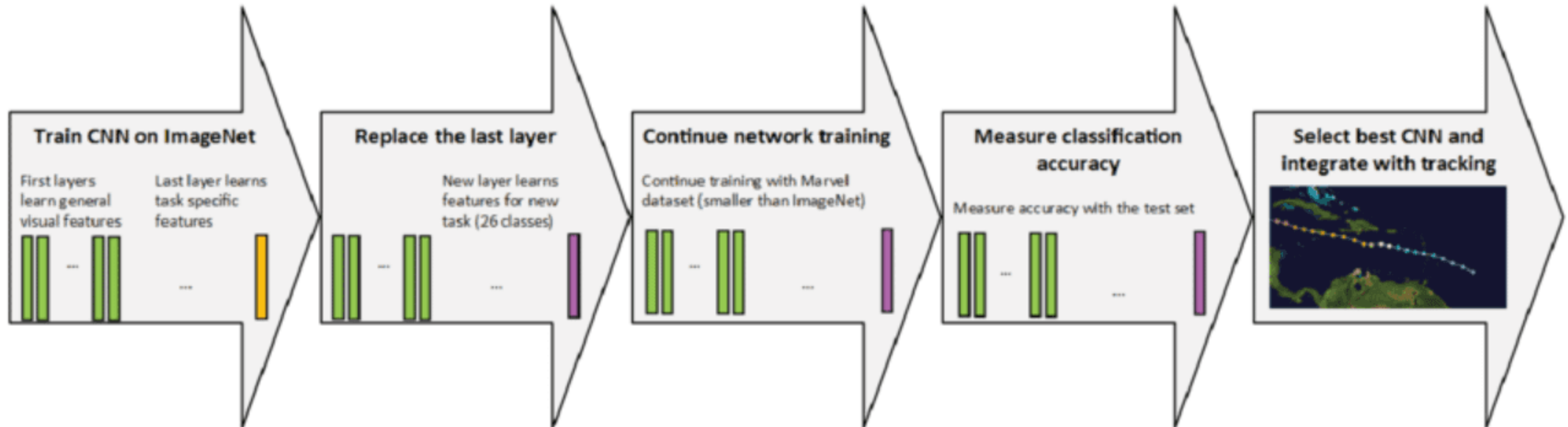
Which to do depends on target task:

- **Freeze:** target task labels are scarce, and we want to avoid overfitting
- **Fine-tune:** target task labels are more plentiful

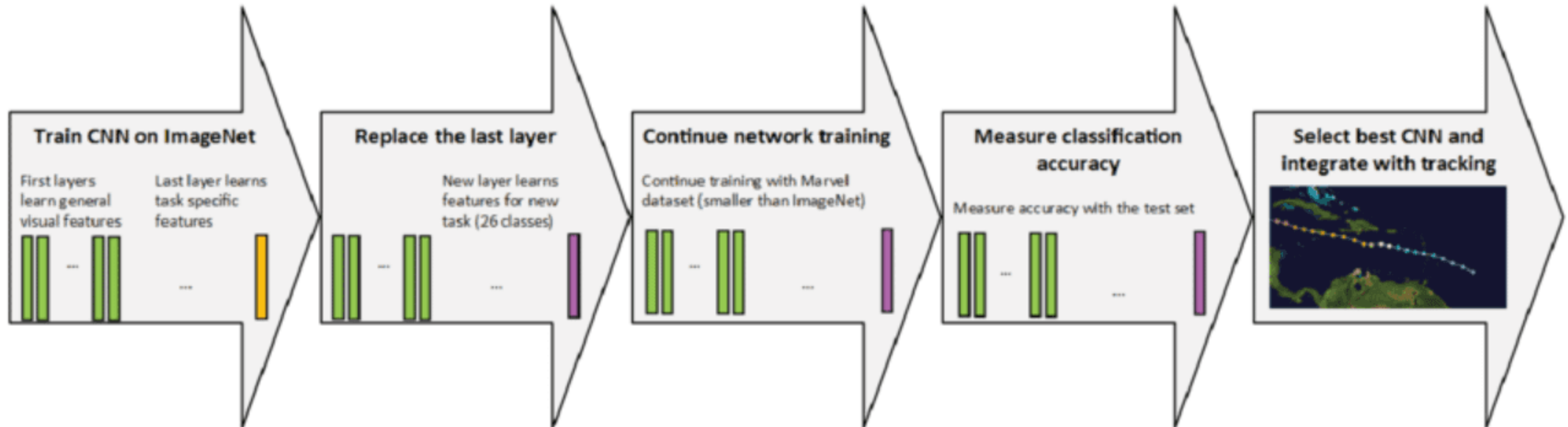
In general, we can set learning rates to be different for each layer to find a tradeoff between freezing and fine tuning



Transfer Learning



Transfer Learning



Implementation

- Transfer learning & fine-tuning
 - https://keras.io/guides/transfer_learning/
- **Transfer Learning with VGG-16 and MobileNet**
 - <https://www.kaggle.com/code/timmate/transfer-learning-with-vgg-16-and-mobilenet>

